

crossfit: A Graph-Based Cross-Fitting Engine for Double/Debiased Machine Learning and Meta-Learners in R

Etienne Peyrot^{*1} and François Petit^{†1}

¹Université Paris Cité and Université Sorbonne Paris Nord, Inserm, INRAE, Center for Research in Epidemiology and Statistics (CRESS), F-75004 Paris, France

Abstract

Cross-fitting is a key ingredient in many semiparametric estimation procedures, such as double/debiased machine learning (DML), enabling valid estimation of low-dimensional targets in the presence of high-dimensional nuisance functions by enforcing out-of-sample use of nuisance predictions. **crossfit** is an R package that provides a general-purpose, estimator-agnostic cross-fitting engine. Users specify (i) a target functional and (ii) a directed acyclic graph (DAG) of nuisance models, with node-specific training fold widths and target-specific evaluation windows. The engine executes a reproducible schedule over folds, panels, and repetitions, returning either a scalar estimate (`mode="estimate"`) or a cross-fitted predictor function for application to new data (`mode="predict"`).

Beyond standard cross-fitting, **crossfit** implements fold-allocation modes that control how training data are shared across nuisance components, including disjoint and independence-enforcing allocations that duplicate reused nodes to reduce dependence between nuisance branches. The implementation targets simulation-heavy benchmarking and method development, with explicit and auditable schedules, defensive validation of specifications and nuisance dependencies, reuse-aware caching to avoid redundant refits, and failure isolation policies for large experiment grids. The **crossfit** package is available on CRAN, openly

^{*}ORCID: 0009-0006-8520-6201

[†]ORCID: 0000-0003-2258-170X

developed on GitHub under GPL-3, and is intended as a lightweight, tested foundation to prototype and empirically evaluate cross-fitted estimators with explicit control over fold geometry, dependence, and computation.

Keywords: cross-fitting; double machine learning; sample splitting; causal inference; semiparametric estimation; meta-learning; R

1 Introduction

Cross-fitting is a sample-splitting method used to reduce bias and recover efficiency in estimators of parameters that depend on nonparametric or high-dimensional estimates of nuisance functions. When nuisance models are both trained and evaluated on the same data, the resulting correlation between the estimation and evaluation steps can introduce estimation errors.

The core idea behind cross-fitting is to use separate samples to estimate the nuisance functions and to evaluate them. Cross-fitting has a long history in semiparametric statistics and is widely used (see, for example, [1, 2, 3, 4, 5, 6], among others). More recently, it has gained significant attention as it is a central component of Double/Debiased Machine Learning (DML) [5] and orthogonal learning [6]. By combining Neyman-orthogonal scores with cross-fitting, these approaches enable the use of machine learning methods to estimate nuisance parameters while still providing semiparametrically efficient and asymptotically normal estimators of the parameter of interest under relatively mild conditions.

The package `crossfit` was developed based on our experience conducting a large benchmark of estimators used in causal inference (Preliminary versions of `crossfit` were used in [7]). Studying the finite-sample properties of cross-fitted estimators requires large simulation experiments in which estimator performance is evaluated across multiple simulated scenarios. Because nuisance models must be repeatedly refitted in each realization, computation becomes a primary bottleneck. Moreover, cross-fitting implementations are susceptible to subtle errors, particularly data leakage and unintended in-sample reuse. For example, when preprocessing steps such as imputation, standardization, or feature selection are fit on the full dataset (rather than within each training fold) and then applied across folds.

Software support for cross-fitting is growing, but many tools focus on a fixed set of estimators and model classes. For example, the `DoubleML` (Python and R) ecosystem provides a high-level interface for canonical causal models and associated inference procedures [8].

In contrast, `crossfit` was developed to support simulation-intensive methodological research and flexible implementation of cross-fitted estimators. The package `crossfit` is designed to (i) make the cross-fitting schedule explicit and auditable, (ii) support a broad variety of user-defined targets and nuisance learners, (iii) handle dependency structures between nuisance components through a DAG abstraction, and (iv) reduce duplicated computation when comparing multiple estimators under shared nuisance components.

This design supports both applied work (implementing cross-fitted estimators safely) and methodological work (systematically studying empirical properties of cross-fitting/double debiasing under different fold geometries, repetitions, and learning strategies).

Statement of need. Cross-fitting is a standard ingredient in modern semi-parametric and causal estimation, nonetheless its empirical behavior (finite-sample stability, sensitivity to fold geometry, number of folds, dependence between nuisance learners, interaction with nuisance complexity) remains an active research topic [9, 10]. A practical barrier to empirical study is that cross-fitting implementations are easy to "get almost right" while silently introducing leakage (e.g., using nuisance predictions trained on the same observations used in target evaluation, or fitting preprocessing steps, such as imputation or standardization, on the full dataset instead of within-fold training data), inconsistent fold schedules across methods, unintended dependence structures between nuisance learners, and fold-specific convergence or fitting failures of nuisance learners (which can occur in large simulation studies and can propagate to the final estimate).

`crossfit` addresses this need by providing:

- A **general interface** for defining nuisance learners and targets without committing to a particular estimator family.
- A **graph-based abstraction** for nuisance dependencies, enabling complex multi-stage pipelines (nuisance-of-nuisance constructions) to be represented explicitly.
- **Explicit control** over fold geometry: node-specific training fold width (`train_fold`), target evaluation window width (`eval_fold`), number of folds K , and repetition count.
- **Well-defined fold-allocation modes** that govern how training folds are assigned across nuisance instances and, crucially, whether dependence is reduced by allocating disjoint training folds and/or duplicating shared nodes (Figure 1).